

Experiment – 1

Name: Aryan

UID: 24BAI70179

Branch: CSE – AIML (CS221)

Section/Group: 24AIT-NTTP-2[A]

Semester: 5th

Date of Performance:

Subject: Full Stack - II

Subject Code: 24CSP-337

EXPERIMENT – 1.1

Aim:

To design and develop a dynamic post composer interface supporting multiple platforms with constraint validation.

Objectives:

- To understand multi-platform content handling.
- To implement real-time validation mechanisms.
- To design responsive and user-friendly UI components.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- State Management: React Hooks / Context API
- Code Editor: Visual Studio Code
- Package Manager: npm
- Browser: Google Chrome / Microsoft Edge

Theory:

Modern social media management tools allow users to compose a single post and publish it across multiple platforms, each of which enforces its own content constraints. For example, a Twitter-like platform may restrict posts to 280 characters, while a platform such as Instagram may require an attached media file and impose different hashtag rules. Designing a composer that adapts to these varying constraints requires a configuration-driven approach, where each platform's rules (character limit, media requirements, hashtag limits, formatting restrictions) are defined separately and applied dynamically based on the platforms selected by the user.

Real-time validation is implemented by attaching change handlers to the text input and media upload controls. As the user types or selects platforms, the composer re-evaluates the content against the active constraints and immediately updates visual feedback elements such as a character counter, warning colour changes, and inline error messages. This gives the user instant awareness of compliance issues before attempting to publish.

To keep the interface modular and reusable, the composer is typically broken down into smaller components, such as a Platform Selector, a Text Editor, a Media Uploader, and a Validation Feedback component, each responsible for a single concern. Responsive design principles ensure that the composer remains usable across desktop, tablet, and mobile screen sizes.

Implementation:

1. Set up a React project and define the component structure.

2. Define a platform constraints configuration object specifying character limits, media rules, and hashtag rules for each platform.
3. Create a Platform Selector component allowing selection of one or more platforms.
4. Create a Post Composer component with a text area for content input.
5. Implement a Media Uploader component for attaching images or videos.
6. Implement real-time validation logic that checks content against the constraints of all selected platforms.
7. Display a live character counter and highlight warnings or errors as constraints are approached or exceeded.
8. Style the composer to be responsive across different screen sizes.
9. Test the composer with different platform combinations and edge-case content.

Code:**• platformConfig.js**

```
export const PLATFORM_LIMITS = {
  Twitter: { maxChars: 280, mediaRequired: false, maxHashtags: 10 },
  Instagram: { maxChars: 2200, mediaRequired: true, maxHashtags: 30 },
  LinkedIn: { maxChars: 3000, mediaRequired: false, maxHashtags: 5 },
};
```

• PostComposer.jsx

```
import { useState, useMemo } from 'react';
import { PLATFORM_LIMITS } from '../platformConfig';

function PostComposer() {
  const [content, setContent] = useState('');
  const [platforms, setPlatforms] = useState([]);
  const [media, setMedia] = useState(null);

  const errors = useMemo(() => {
    return platforms.map((p) => {
      const limit = PLATFORM_LIMITS[p];
      const issues = [];
      if (content.length > limit.maxChars) {
        issues.push(`${p}: exceeds ${limit.maxChars} characters`);
      }
      if (limit.mediaRequired && !media) {
        issues.push(`${p}: media attachment required`);
      }
      return { platform: p, issues };
    });
  }, [content, platforms, media]);

  return (
    <div className="composer">
      <textarea value={content} onChange={(e) => setContent(e.target.value)} />
      <p>{content.length} characters</p>
      {errors.map((e) =>
        e.issues.map((msg, i) => <p key={i} className="error">{msg}</p>)
      )}
    </div>
  );
}

export default PostComposer;
```

• CharacterCounter.jsx

```
function CharacterCounter({ current, max }) {
  const remaining = max - current;
  const isWarning = remaining < max * 0.1;
  const isError = remaining < 0;

  return (
    <span className={isError ? 'count-error' : isWarning ? 'count-warning' : 'count-ok'}>
      {current} / {max}
    </span>
  );
}

export default CharacterCounter;
```

Output:

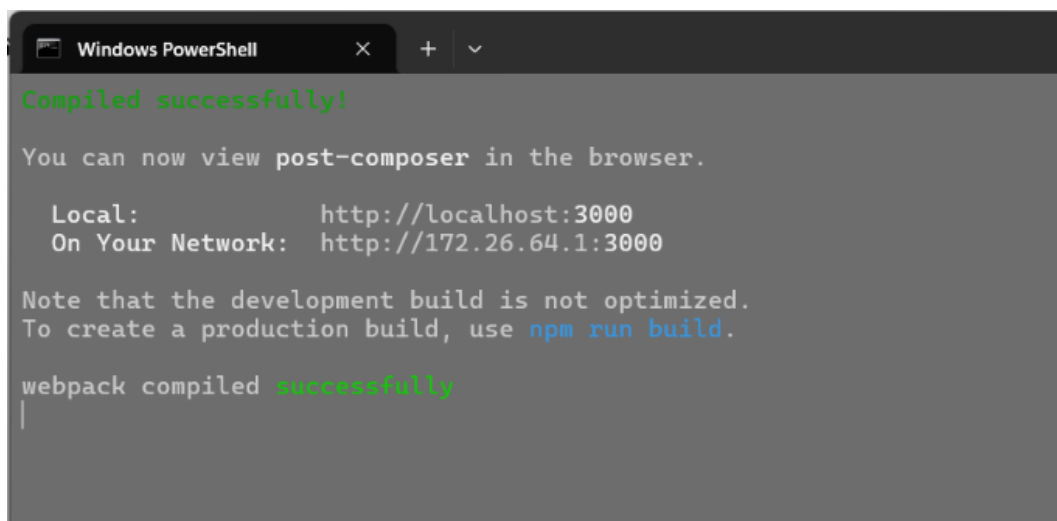


Fig A: Post Composer with Platform Selector

Post Composer

Select Platform:

Twitter

Write Your Post:

ARYAN
24BAI70179
24NTPP 2A

Characters: 28 / 280

✓ Ready to post

Save Draft

Fig B: Real-Time Validation Warnings

Learning Outcomes:

- Understand how to handle varying content constraints across multiple platforms.
- Implement real-time, configuration-driven validation logic.



- Design modular and reusable UI components.
- Provide clear visual feedback to improve user experience.
- Build a responsive interface that works across device sizes.

Conclusion:

A dynamic post composer was successfully designed and implemented to support multiple social media platforms, each with distinct content constraints. Real-time validation and visual feedback mechanisms, such as character counters and warning indicators, were incorporated to guide users toward compliant content before publishing. The modular, configuration-driven design allows new platforms and rules to be added with minimal changes to the core composer logic.

EXPERIMENT – 1.2

Aim:

To implement a draft management system that allows users to save, retrieve, and manage post drafts within the frontend, with optional simulation of backend interactions.

Objectives:

- To manage draft data using frontend state.
- To implement CRUD operations for draft handling.
- To understand asynchronous workflows in UI applications.
- To optionally simulate backend communication using mock APIs.

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Frontend: React.js
- State Management: useState / useReducer / Context API
- Mock API Simulation: JavaScript Promises with simulated network delay
- Code Editor: Visual Studio Code
- Package Manager: npm
- Browser: Google Chrome / Microsoft Edge

Theory:

A draft management system allows users to save partially completed posts and return to them later without losing their work. On the frontend, drafts can be held in local component state or in a centralized store managed with useReducer or Context API, depending on the complexity of the application. Each draft is typically modeled as an object containing an identifier, title, content, target platform(s), and a last-modified timestamp.

Managing drafts requires implementing the four fundamental CRUD operations: Create (saving a new draft), Read (loading a draft back into the composer or listing all drafts), Update (modifying an existing draft), and Delete (removing a draft the user no longer needs). Because real applications eventually persist this data to a server, these operations are commonly written as asynchronous functions, even when working purely with local state, so that the code can later be connected to a real backend with minimal changes.

To simulate backend communication without an actual server, a mock API layer can be created using JavaScript Promises combined with setTimeout to imitate network latency, and can optionally simulate failures to test error handling. This allows the UI's loading states, error states, and optimistic updates to be developed and tested independently of backend availability.

Implementation:

1. Define a Draft data model containing id, title, content, platform, and lastModified fields.
2. Set up frontend state (useState or useReducer) to hold the list of drafts.
3. Implement a mock API module that simulates asynchronous save, fetch, and delete operations with a network delay.
4. Implement a 'Save Draft' action that creates a new draft or updates an existing one.
5. Implement a 'Load Draft' action that retrieves a draft and populates the composer with its content.

6. Implement a 'Delete Draft' action that removes a draft from state.
7. Add loading and error states around each asynchronous operation.
8. Display the list of saved drafts with edit and delete controls.
9. Test the CRUD workflow end-to-end and verify state remains consistent after each operation.

Code:**• mockApi.js**

```
let drafts = [];  
let nextId = 1;  
  
function delay(ms) {  
  return new Promise((resolve) => setTimeout(resolve, ms));  
}  
  
export async function saveDraft(draft) {  
  await delay(500);  
  if (draft.id) {  
    drafts = drafts.map((d) => (d.id === draft.id ? { ...draft, lastModified: Date.now() } : d));  
  } else {  
    draft = { ...draft, id: nextId++, lastModified: Date.now() };  
    drafts.push(draft);  
  }  
  return draft;  
}  
  
export async function fetchDrafts() {  
  await delay(500);  
  return [...drafts];  
}  
  
export async function deleteDraft(id) {  
  await delay(300);  
  drafts = drafts.filter((d) => d.id !== id);  
  return id;  
}
```

• draftReducer.js

```
export function draftReducer(state, action) {  
  switch (action.type) {  
    case 'LOADING':  
      return { ...state, loading: true, error: null };  
    case 'SET_DRAFTS':  
      return { ...state, loading: false, drafts: action.payload };  
    case 'UPSERT_DRAFT': {  
      const exists = state.drafts.some((d) => d.id === action.payload.id);  
      const drafts = exists  
        ? state.drafts.map((d) => (d.id === action.payload.id ? action.payload : d))  
        : [...state.drafts, action.payload];  
      return { ...state, loading: false, drafts };  
    }  
    case 'REMOVE_DRAFT':  
      return { ...state, loading: false, drafts: state.drafts.filter((d) => d.id !== action.payload) };  
    case 'ERROR':  
      return { ...state, loading: false, error: action.payload };  
    default:  
      return state;  
  }  
}
```

• DraftList.jsx

```
import { useEffect, useReducer } from 'react';
import { draftReducer } from './draftReducer';
import { fetchDrafts, deleteDraft } from './mockApi';

function DraftList({ onEdit }) {
  const [state, dispatch] = useReducer(draftReducer, { drafts: [], loading: false, error: null });

  useEffect(() => {
    dispatch({ type: 'LOADING' });
    fetchDrafts()
      .then((data) => dispatch({ type: 'SET_DRAFTS', payload: data }))
      .catch((err) => dispatch({ type: 'ERROR', payload: err.message }));
  }, []);

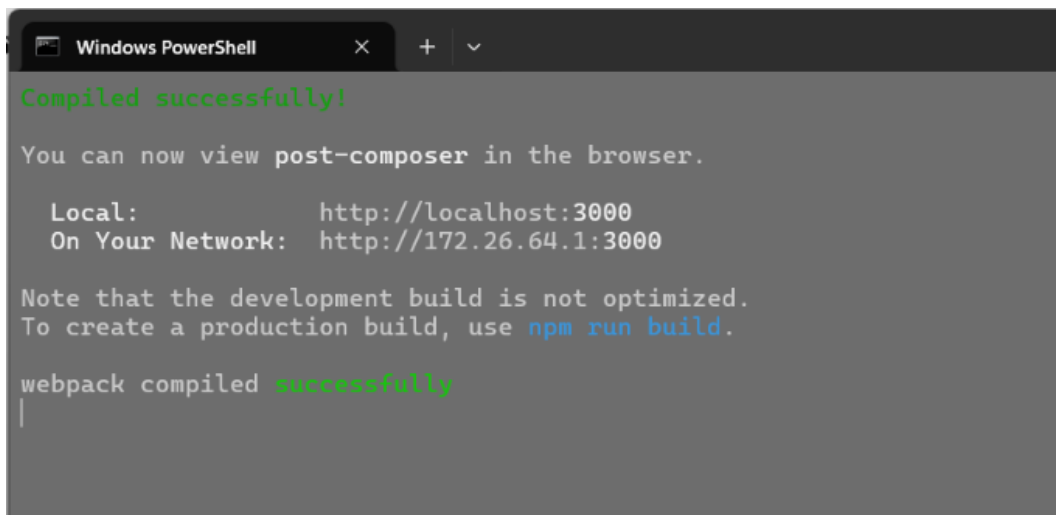
  const handleDelete = async (id) => {
    await deleteDraft(id);
    dispatch({ type: 'REMOVE_DRAFT', payload: id });
  };

  if (state.loading) return <p>Loading drafts...</p>;
  if (state.error) return <p>Error: {state.error}</p>;

  return (
    <ul>
      {state.drafts.map((d) => (
        <li key={d.id}>
          {d.title}
          <button onClick={() => onEdit(d)}>Edit</button>
          <button onClick={() => handleDelete(d.id)}>Delete</button>
        </li>
      ))}
    </ul>
  );
}

export default DraftList;
```

Output:



```
Windows PowerShell
Compiled successfully!

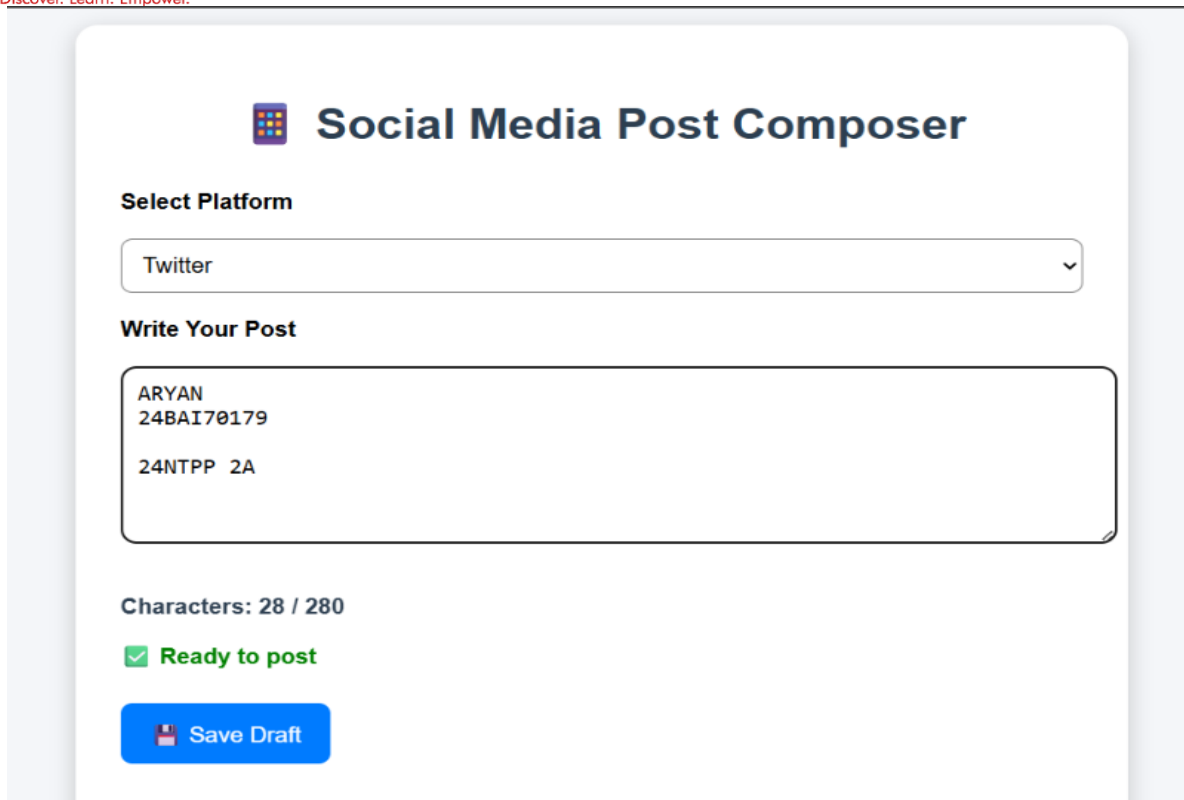
You can now view post-composer in the browser.

Local:      http://localhost:3000
On Your Network:  http://172.26.64.1:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
|
```

Fig A: Saved Drafts List



The image shows a web interface for a 'Social Media Post Composer'. At the top, there's a title 'Social Media Post Composer' with a small icon. Below it, a 'Select Platform' dropdown menu is set to 'Twitter'. Underneath is a 'Write Your Post' section with a text area containing the text: 'ARYAN', '24BAI70179', and '24NTPP 2A'. Below the text area, it says 'Characters: 28 / 280'. There's a green checkmark and the text 'Ready to post'. At the bottom, there's a blue button with a floppy disk icon and the text 'Save Draft'.

Fig B: Loading State During Simulated API Call

Learning Outcomes:

- Manage structured draft data using frontend state.
- Implement Create, Read, Update, and Delete operations on the frontend.
- Understand and implement asynchronous workflows using Promises and async/await.
- Simulate backend communication using a mock API layer.
- Handle loading and error states gracefully in a user interface.

Conclusion:

A draft management system was successfully implemented, enabling users to save, retrieve, update, and delete post drafts entirely on the frontend. Asynchronous CRUD operations were simulated using a mock API layer with artificial network delay, allowing loading and error states to be developed and tested realistically. This design keeps the draft workflow decoupled from any specific backend, making it straightforward to connect to a real API in the future.